

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software and Automation Engineering

Tacu Loredana

FAF-242

REPORT

Laboratory Work No.2

Meet the OS You Will Build

Track B: build your own small OS (xv6)

Checked by:

_____, PhD

Department of Software and Automation Engineering,

Faculty of Computers, Informatics and Microelectronics,

UTM **Chisinau – 2026**

1 Purpose of the Laboratory Work

The goal of this laboratory work is to build, boot, and extend xv6 – a small, real, bootable Unix (a modern re-implementation of Unix Version 6, in about 9,000 lines of C, running on RISC-V under the QEMU emulator). Unlike Lab 1, where the OS could only be observed from the outside, xv6 is small enough to read both sides of every interaction: the user program that asks, and the kernel code that answers.

2 Problem Conditions

Requirements and Conditions:

(1) **Part 1 – Build and boot:** compile the kernel and user programs with `make qemu` and boot xv6 in the emulator.

(2) **Part 2 – Use xv6 as the Unix it is:** run xv6's built-in programs (`ls`, `cat`, `echo`, pipes, `wc`) at its shell prompt.

(3) **Part 3 – Read the source:** read a user program (`user/ls.c`, `user/cat.c`) and locate the kernel-side implementation of the system calls it uses.

(4) **Part 4 – Write the first xv6 program:** implement `user/sleep.c`, a program that pauses for a number of clock ticks via a system call, add it to the build, and test it.

(5) Two versions of the deliverable are required: the working `user/sleep.c` and `notes.md` with the answers to the "Observe" questions from Parts 2 and 3.

3 Part 1 – Build It and Boot It

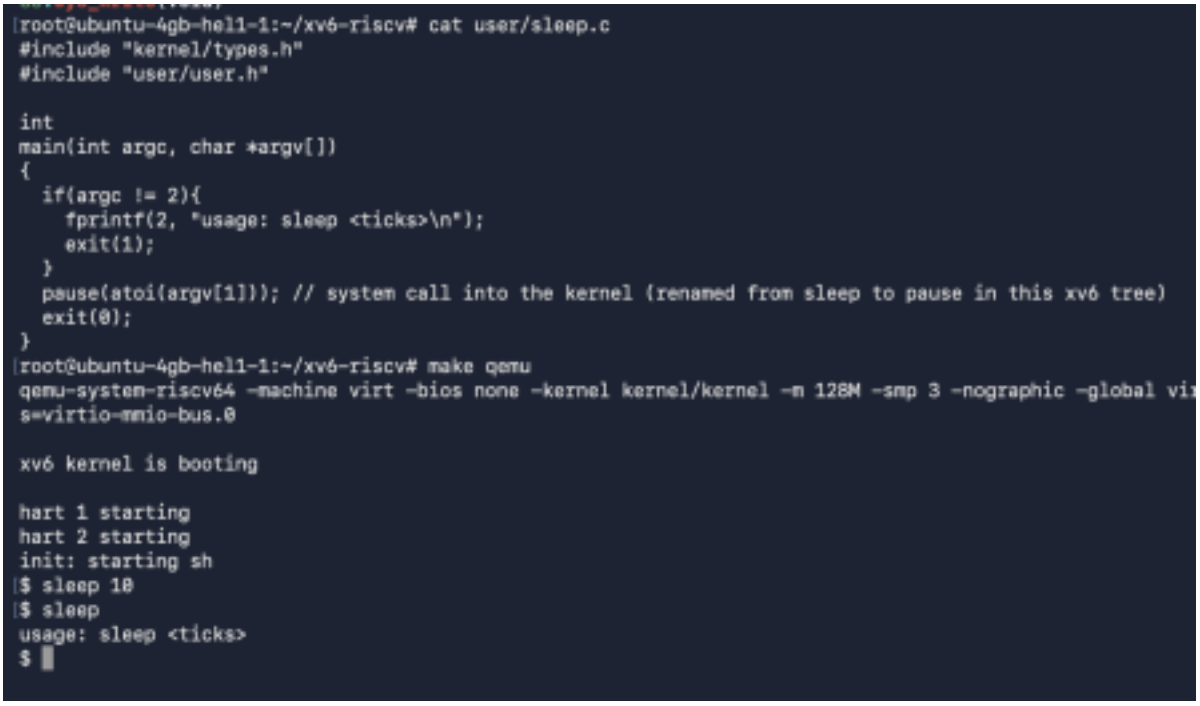


Figure 1: make qemu building and booting xv6 (this screenshot also covers Part 4's test, below).

The kernel compiles, boots on QEMU, and drops into the xv6 shell (`init: starting sh` followed by the `$` prompt).

4 Part 2 – Use xv6 as the Unix It Is

```
$ ls
cat README
echo hello xv6
ls | grep c
wc README.              1 1 1024
..              1 1 1024
README          2 2 2441
cat             2 3 35376
echo            2 4 34312
sleep           2 5 34032
forktest        2 6 17280
grep            2 7 42736
init            2 8 34768
kill            2 9 34240
ln              2 10 34032
ls              2 11 41520
mkdir           2 12 34280
rm              2 13 34264
sh              2 14 56136
stressfs        2 15 35160
usertests       2 16 200056
grind           2 17 50504
wc              2 18 36336
zombie          2 19 33648
logstress       2 20 36184
forphan         2 21 35048
dorphan         2 22 34488
sync            2 23 33712
console         3 24 0
$ xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6).  xv6 loosely follows the structure and style of v6,
but is implemented for a modern RISC-V multiprocessor using ANSI C.

ACKNOWLEDGMENTS

xv6 is inspired by John Lions's Commentary on UNIX 6th Edition (Peer
to Peer Communications; ISBN: 1-57398-013-7; 1st edition (June 14,
2000)).  See also https://pdos.csail.mit.edu/6.1810/, which provides
pointers to on-line resources for v6.

The following people have made contributions: Russ Cox (context switching,
locking), Cliff Frey (MP), Xiao Yu (MP), Nickolai Zeldovich, and Austin
Clements.

We are also grateful for the bug reports and patches contributed by
Abhinavpatel00, Takahiro Aoyagi, Marcelo Arroyo, Hirbod Behnam, Silas
Boyd-Wickizer, Anton Burtsev, carlclone, Ian Chen, clivezeng, Dan
Cross, Cody Cutler, Mike CAT, Tej Chajed, Asami Doi,Wenyang Duan,
echtwerner, eyalz800, Nelson Elhage, Saar Ettinger, Alice Ferrazzi,
Nathaniel Filardo, flespark, Peter Froehlich, Yakir Goaron, Shivam
Handa, Matt Harvey, Bryan Henry, jaichenhengjie, Jim Huang, Matúš
Jókay, John Jolly, Alexander Kapshuk, Anders Kaseorg, kehao95,
Wolfgang Keller, Jungwoo Kim, Jonathan Kimmitt, Eddie Kohler, Vadim
Kolontsov, Austin Liew, Bruce Lowekamp, l0stman, Pavan Maddamsetti,
Imbar Marinescu, Yandong Mao, Matan Shabtay, Hitoshi Mitake, Carmi
Merimovich, mes900903, Mark Morrissey, mtasm, Joel Nider, Hayato Ohhashi,
```

Figure 2: ls, cat README inside xv6.

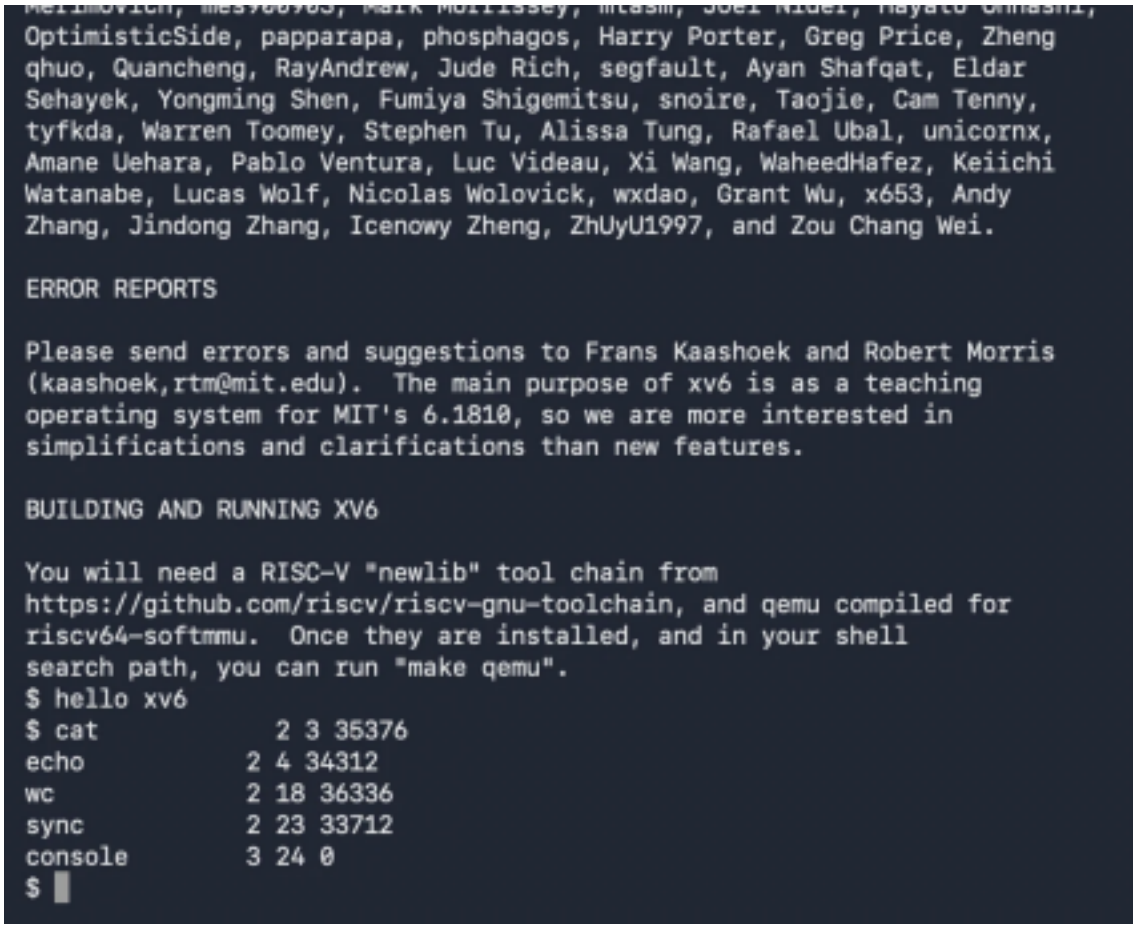


Figure 3: echo hello xv6, ls | grep c, wc README.

Observe

(1) **Three programs xv6 ships with:** cat, echo, wc (also present: ls, grep, sh, kill, mkdir, ln, rm, usertests, and more – the full set is visible in the ls listing above).

(2) **Which two OS features must exist for a pipe to work:** (a) **process manage ment** – the shell must be able to create two separate processes (via fork) to run ls and grep concurrently, and (b) **inter-process communication** – the kernel must provide a pipe() mechanism: a kernel-buffered channel connecting one process's standard out put to another process's standard input, so bytes written by ls can be read by grep without either program knowing about the other directly.

(3) **xv6 shell vs. the Linux shell from Lab 1:** the xv6 shell (sh) supports the same basic Unix conventions as bash – running commands, redirection, and pipes – but it is a tiny, minimal implementation with almost none of bash's extra features (no job control, no scripting constructs, a handful of built-ins), which is exactly why its source is short enough to read in full.

5 Part 3 – Read the Source

```
root@ubuntu-4gb-hel1-1:~/xv6-riscv# sed -n "1,40p" user/ls.c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

char *
fmtname(char *path)
{
  static char buf[DIRSIZ + 1];
  char *p;

  // Find first character after last slash.
  for (p = path + strlen(path); p >= path && *p != '/'; p--)
    ;
  p++;

  // Return blank-padded name.
  if (strlen(p) >= DIRSIZ)
    return p;
  memmove(buf, p, strlen(p));
  memset(buf + strlen(p), ' ', DIRSIZ - strlen(p));
  buf[sizeof(buf) - 1] = '\0';
  return buf;
}

void
ls(char *path)
{
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if ((fd = open(path, O_RDONLY)) < 0) {
    fprintf(2, "ls: cannot open %s\n", path);
    return;
  }

  if (fstat(fd, &st) < 0) {
root@ubuntu-4gb-hel1-1:~/xv6-riscv#
```

Figure 4: sed -n "1,40p" user/ls.c.

```
root@ubuntu-4gb-hel1-1:~/xv6-riscv# sed -n "1,30p" user/cat.c
#include "kernel/types.h"
#include "kernel/fcntl.h"
#include "user/user.h"

char buf[512];

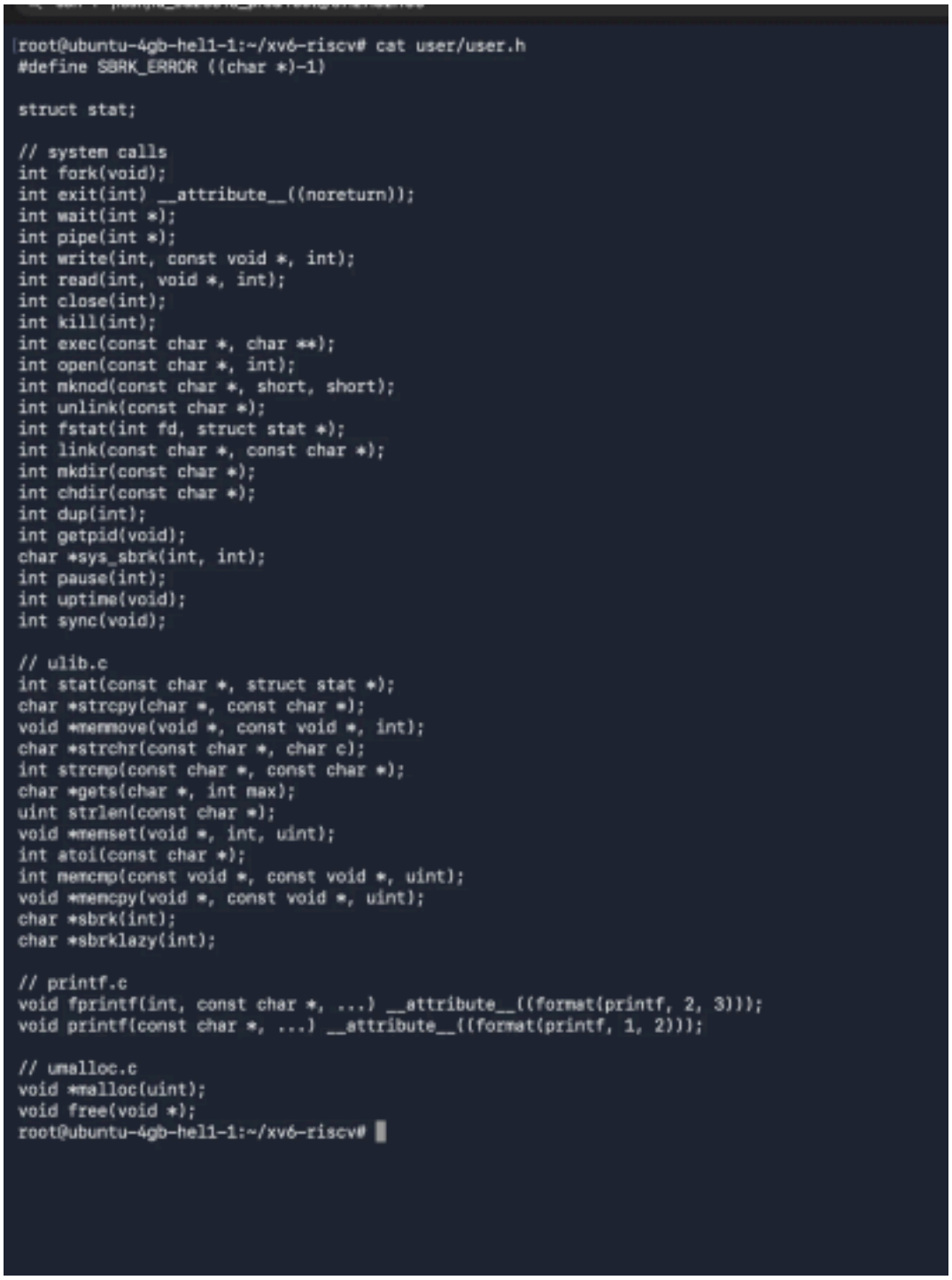
void
cat(int fd)
{
  int n;

  while ((n = read(fd, buf, sizeof(buf))) > 0) {
    if (write(1, buf, n) != n) {
      fprintf(2, "cat: write error\n");
      exit(1);
    }
  }
  if (n < 0) {
    fprintf(2, "cat: read error\n");
    exit(1);
  }
}

int
main(int argc, char *argv[])
{
  int fd, i;

  if (argc <= 1) {
    cat(0);
root@ubuntu-4gb-hel1-1:~/xv6-riscv#
```

Figure 5: sed -n "1,30p" user/cat.c.



```
root@ubuntu-4gb-hel1-1:~/xv6-riscv# cat user/user.h
#define SBRK_ERROR ((char *)-1)

struct stat;

// system calls
int fork(void);
int exit(int) __attribute__((noreturn));
int wait(int *);
int pipe(int *);
int write(int, const void *, int);
int read(int, void *, int);
int close(int);
int kill(int);
int exec(const char *, char **);
int open(const char *, int);
int mknod(const char *, short, short);
int unlink(const char *);
int fstat(int fd, struct stat *);
int link(const char *, const char *);
int mkdir(const char *);
int chdir(const char *);
int dup(int);
int getpid(void);
char *sys_sbrk(int, int);
int pause(int);
int uptime(void);
int sync(void);

// ulib.c
int stat(const char *, struct stat *);
char *strcpy(char *, const char *);
void *memmove(void *, const void *, int);
char *strchr(const char *, char c);
int strcmp(const char *, const char *);
char *gets(char *, int max);
uint strlen(const char *);
void *memset(void *, int, uint);
int atoi(const char *);
int memcmp(const void *, const void *, uint);
void *memcpy(void *, const void *, uint);
char *sbrk(int);
char *sbrklazy(int);

// printf.c
void fprintf(int, const char *, ...) __attribute__((format(printf, 2, 3)));
void printf(const char *, ...) __attribute__((format(printf, 1, 2)));

// umalloc.c
void *malloc(uint);
void free(void *);
root@ubuntu-4gb-hel1-1:~/xv6-riscv#
```

Figure 6: cat user/user.h – the full list of system calls available to xv6 programs.



Figure 7: grep -n "sys_read\|sys_write" kernel/sysfile.c.

Observe

(1) **Which system calls does user/cat.c use, and what does each ask the kernel to do:**

- read(fd, buf, sizeof(buf)) – asks the kernel to copy up to sizeof(buf) bytes from the file/pipe behind file descriptor fd into buf.
- write(1, buf, n) – asks the kernel to copy n bytes from buf out to file descriptor 1 (standard output).
- exit(1) – asks the kernel to terminate the calling process (used here on read/write errors).

(2) **Where sys_read is implemented:** kernel/sysfile.c, line 69 (sys_write is right below it, at line 83).

(3) **Difference between the code in kernel/ and the code in user/:** kernel/ is privileged code – it runs in supervisor mode with direct access to hardware and manages shared resources (the scheduler, memory, the file system, and the handlers behind every system call); user/ holds ordinary, unprivileged programs that can only ask for those resources indirectly, by making system calls that trap into the kernel.

6 Part 4 – Write the First xv6 Program

Note: this course's xv6 tree renamed the sleep syscall to pause – confirmed in user/user.h, usys.pl, and used throughout usertests.c – so the working program calls pause(), not sleep() as the handout's literal snippet shows.

user/sleep.c:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);

  }
```

```
    pause(atoi(argv[1])); // syscall into the kernel (renamed sleep -> pause) exit(0);
}
```

Added $U/_sleep\ to UPROGS in the Makefile, rebuilt, and tested (see Figure 1 above, which shows both the boot and this test in one session):

```
$ sleep 10
$ sleep
usage: sleep <ticks>
$
```

sleep 10 paused for 10 ticks and returned to the prompt with no output; bare sleep correctly printed the usage message and exited with status 1.

7 Conclusion and Critical Analysis

Summary of Work:

- Cloned, built, and booted xv6 on QEMU (make qemu).
- Exercised xv6's built-in Unix programs, including a pipe between ls and grep.
- Read user/ls.c and user/cat.c, and traced their system calls to the kernel-side han dlers in kernel/sysfile.c.
- Implemented, built, and tested a new user program, sleep, that makes a real system call into a self-compiled kernel.

Critical Analysis:

- **Renamed syscalls break naive copy-paste:** this course's xv6 tree renamed sleep to pause everywhere (user.h, usys.pl, usertests.c). Blindly pasting the handout's literal sleep(atoi(argv[1])) would have failed to compile; verifying against user/user.h first caught this before it became a build error.
- **The xv6/Linux prompt boundary is easy to lose track of:** xv6's shell ($) and the host Linux shell (root@...#) look superficially similar, but only one of them has access to the C source tree and make. Commands typed into the wrong one fail in confusing, shell-specific ways (e.g. sed inside xv6 has no such program, so it reports "exec failed").
- **The kernel/user split is the organizing idea of the whole codebase:** once sys_read/sys_write were located in kernel/sysfile.c, it became clear that every "or dinary" library call in user/ ('read', 'write', ...) is really just a thin trap into a handful of functions on the other side of the privilege boundary.

Overall, this laboratory work demonstrates how to read, build, and extend a real (if tiny) operating system from both sides of the system-call boundary – something impossible with Lab 1's Linux VM, where only the outside of the OS could be observed.